

Swarm

Arturo Silvelo

Try New Roads

Swarm

Swarm

Docker Swarm es la **solución nativa de orquestación** de Docker que permite gestionar un cluster de motores Docker como un **sistema virtual único**.

Elementos:

- **Nodo:** máquina que participa en el clúster.
- **Servicio:** Contenedor y réplicas.
- **Stack:** conjunto de servicios desplegados y gestionados juntos.

Características principales

- **Gestión de cluster integrada** - sin software adicional
- **Escalado automático** - ajusta réplicas según demanda
- **Reconciliación del estado deseado** - auto-reparación
- **Service discovery + Balanceo de carga** - DNS y distribución automática
- **Rolling updates** - despliegues sin downtime con rollback
- **Seguro por defecto** - TLS

Gestión Swarm

Estructura Swarm

Un swarm se compone de nodos manager y nodos worker. Los managers son responsables de gestionar el estado del cluster, tomar decisiones de orquestación y proporcionar las APIs del swarm - siempre debe haber al menos uno y se recomienda un número impar para garantizar el consenso. Los workers ejecutan las tareas (contenedores) que les asignan los managers.

Añadir al swarm

Cuando creas un swarm, inicias con un solo Docker Engine en modo Swarm. Para aprovechar todas las ventajas de Swarm, puedes añadir más nodos:

- **Añadir nodos worker** aumenta la capacidad del cluster. Los servicios se distribuyen entre todos los nodos disponibles, sean workers o managers, permitiendo escalar el swarm sin afectar el consenso de los managers.
- **Añadir nodos manager** incrementa la tolerancia a fallos. Los managers gestionan la orquestación y el estado del cluster. Entre ellos, uno actúa como líder. Si el líder falla, los managers restantes eligen un nuevo líder y el cluster sigue funcionando.

Swarm con DinD

- Crear el nodo manager

```
docker run -d --privileged --name manager --network host docker:dind
```

- Crear nodos

```
docker run -d --privileged --name worker-1 --hostname worker-1 docker:dind  
docker run -d --privileged --name worker-2 --hostname worker-1 docker:dind
```

- Iniciar swarm

```
docker exec manager docker swarm init --advertise-addr <ip|interface>
```

Añadir nodos **worker**

Para obtener el comando de unión, incluyendo el token para nodos worker:

```
docker swarm join-token worker
```

El comando **swarm join** une los nodos al swarm y realiza las siguientes acciones:

- Extiende la red overlay
- Solicita un certificado TLS

```
docker exec worker-1 docker swarm join ....  
docker exec worker-2 docker swarm join ....
```

Añadir nodos **manager**

Para obtener el comando de unión, incluyendo el token para nodos manager:

```
docker swarm join-token manager
```

```
docker exec manager-2 docker swarm join ....
```

Gestionar nodos

Para ver todos los nodos que forman parte del swarm, ejecuta el siguiente comando desde un nodo manager:

```
$docker node ls
docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER	STATUS	ENGINE	VERSION
jijwfjp1atvi3ho32puxax1pe *	silvelo	Ready	Active	Leader		28.5.0	
gi7u04ihrvr95dn6ei1144vyh	worker-1	Ready	Active			28.5.0	
l5lddhqxecbkhq1ue578z9c6x	worker-2	Ready	Active			28.5.0	

- **AVAILABILITY:** Indica si el planificador puede asignar tareas al nodo:
 - **Active:** El nodo puede recibir tareas nuevas.
 - **Pause:** No recibe tareas nuevas, pero las existentes siguen ejecutándose.
 - **Drain:** No recibe tareas nuevas y las existentes se reubican en otros nodos.

- **MANAGER STATUS:** Muestra la participación del nodo en el consenso Raft:
 - *(Vacío)*: Nodo worker, no participa en la gestión del swarm.
 - **Leader**: Nodo manager principal, toma decisiones de orquestación.
 - **Reachable**: Manager participante en el quorum Raft, elegible como nuevo líder si el actual falla.
 - **Unavailable**: Manager que no puede comunicarse con otros managers; es recomendable añadir o promover otro manager.

Cambiar Estado

Cambiar el parámetro `availability` de un nodo en Docker Swarm sirve para controlar si ese nodo puede recibir nuevas tareas o no.

```
$docker node update --availability pause|drain|active worker-1
```

```
$docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER	STATUS	ENGINE	VERSION
jijwfjp1atvi3ho32puxax1pe *	silvelo	Ready	Active	Leader		28.5.0	
gi7u04ihrvr95dn6ei1144vyh	worker-1	Ready	Pause			28.5.0	
l5lddhqxecbkhq1ue578z9c6x	worker-2	Ready	Active			28.5.0	

Promover o degradar un nodo

Puedes **promover** un nodo worker a manager, lo cual es útil si un manager queda fuera de servicio o si necesitas más managers para mantener el quorum.

```
$docker node promote worker-1 worker-2
```

```
$docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
jijwfjp1atvi3ho32puxax1pe *	silvelo	Ready	Active	Leader	28.5.0
gi7u04ihrvr95dn6ei1144vyh	worker-1	Ready	Active	Reachable	28.5.0
l5lddhqxecbkhq1ue578z9c6x	worker-2	Ready	Active	Reachable	28.5.0

De igual forma, puedes **degradar** (demote) un manager a worker, por ejemplo, para realizar tareas de mantenimiento o reducir el número de managers.

```
$docker node demote worker-1 worker-2
```

```
$docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
jijwfjp1atvi3ho32puxax1pe *	silvelo	Ready	Active	Leader	28.5.0
gi7u04ihrvr95dn6ei1144vyh	worker-1	Ready	Active		28.5.0
l5lddhqxecbkhq1ue578z9c6x	worker-2	Ready	Active		28.5.0

Eliminar nodo

En ocasiones es necesario eliminar un nodo del swarm, por ejemplo, cuando un nodo deja de estar disponible, se va a dar de baja definitivamente o simplemente quieres reorganizar el cluster.

- Nodo abandone el swarm

```
docker exec worker-2 docker swarm leave
```

- Eliminar nodo del swarm

```
docker exec manager docker node rm worker-2
```

Para nodos manager primero hay que degradar el nodo

Servicios

Crear

Para desplegar una aplicación en el cluster Swarm, se utiliza el concepto de **servicio**. Un servicio define la imagen del contenedor, el número de réplicas y otras opciones de despliegue.

```
docker service create --name single ghcr.io/trynewroads/docker-course-advance:latest
```

```
docker service ls
```

Actualizar

Puedes modificar casi cualquier aspecto de un servicio existente usando el comando `docker service update`. Al actualizar un servicio, Docker detiene sus contenedores y los reinicia con la nueva configuración.

```
docker service update --publish-add 3000:3000 single
```

Esalar

Puedes aumentar o disminuir el número de réplicas de un servicio en Swarm de forma sencilla usando el comando `docker service scale`.

```
docker service scale single=5
```

Esto ajusta el servicio `single` para que tenga 5 réplicas activas distribuidas entre los nodos del cluster.

Eliminar

Para eliminar un servicio en Docker Swarm y detener todas sus tareas y contenedores asociados.

```
docker service rm single
```

Esto elimina el servicio `single` y libera los recursos en todos los nodos del cluster.